PRACTICAL-08

1.

AIM:

Guarded calculator: read two numbers and an operator; throw a custom

DivideByZeroException and catch invalid-number input separately, printing a clear

message each time; loop until a valid calculation succeeds and use finally to log each attempt.

CODE:

```
import java.util.Scanner;
import java.util.InputMismatchException;

class DivideByZeroException extends Exception {
DivideByZeroException(String message) {
super(message);
    }
}

public class GuardedCalculator {

    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
boolean success = false;        int attempt =
0;
```

```
        while (!success) {
attempt++;

            try {
                System.out.print("Enter first number: ");
double a = sc.nextDouble();

                System.out.print("Enter operator (+, -, *, /): ");
                String op = sc.next();

                System.out.print("Enter second number: ");
double b = sc.nextDouble();

                double result;

                switch (op) {
case "+":
result = a + b;
break;

                    case "-":
result = a - b;
break;

                    case "*":
                        result = a * b;
                        break;
```

```
                case "/":
                    if (b == 0) {
                        throw new DivideByZeroException(
                            "Cannot divide by zero!"
                        );
}                   result = a /
b;                      break;

                default:
                    System.out.println("Invalid operator!");
continue;
                }

                System.out.println("Result: " + result);
success = true;

            } catch (DivideByZeroException e) {
                System.out.println(e.getMessage());

            } catch (InputMismatchException e) {
System.out.println("Invalid number input!");

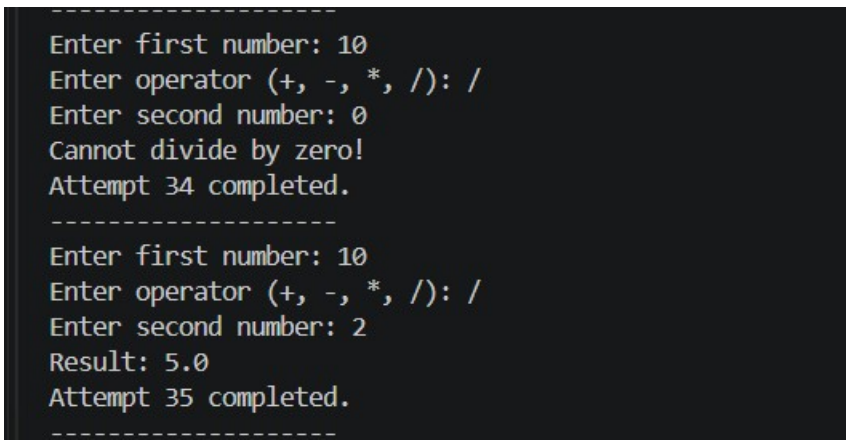
                if (sc.hasNext()) {
sc.next();
                }

            } finally {
```

```
            System.out.println("Attempt " + attempt + " completed.");
            System.out.println("--------------------");
        }
    }

    sc.close();
  }
}
```

OUTPUT:



2.

AIM:

Stock issue: a Warehouse where issue(item, qty) throws a custom checked

OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0.

Process a list of requests, catching and reporting each failure without stopping the run.

CODE:

```
import java.util.Scanner;


class OutOfStockException extends Exception {
```

```
    private int shortfall;

    OutOfStockException(int shortfall) {
super("Out of stock! Shortfall: " + shortfall);
this.shortfall = shortfall;
    }

    public int getShortfall() {
return shortfall;
    }
}

class InvalidQuantityException extends Exception {
InvalidQuantityException(String message) {
super(message);
    }
}

class Warehouse {
    private int stock;

    Warehouse(int stock) {
this.stock = stock;
    }
    void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {
```

```
        if (qty <= 0) {
            throw new InvalidQuantityException(
                "Invalid quantity! Quantity must be greater than zero."
            );
        }

        if (qty > stock) {
            throw new OutOfStockException(qty - stock);
        }

        stock -= qty;
        System.out.println("Issued " + qty + " units of " + item);
        System.out.println("Remaining stock: " + stock);
    }

    int getStock() {
return stock;
    }
}

public class WarehouseDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Warehouse warehouse = new Warehouse(20);

        System.out.print("Enter number of requests: ");
int n = sc.nextInt();         sc.nextLine();
```

```
        for (int i = 1; i <= n; i++) {
            System.out.println("\nRequest " + i);

            System.out.print("Enter item name: ");
            String item = sc.nextLine();

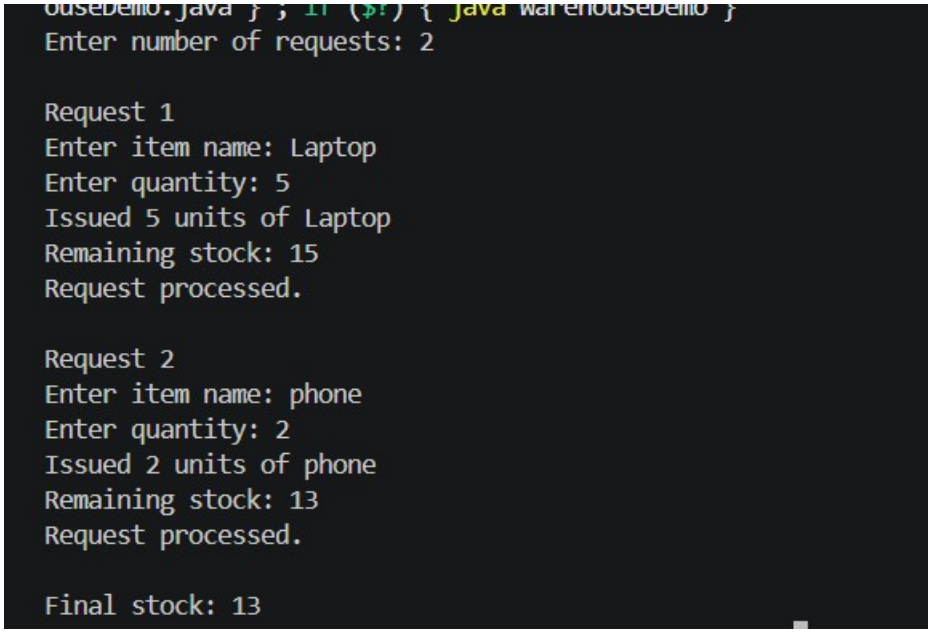
            System.out.print("Enter quantity: ");
int qty = sc.nextInt();             sc.nextLine();

            try {
                warehouse.issue(item, qty);
            } catch (OutOfStockException e) {
                System.out.println(e.getMessage());
            } catch (InvalidQuantityException e) {
                System.out.println(e.getMessage());
            }

            System.out.println("Request processed.");
        }
        System.out.println("\nFinal stock: " + warehouse.getStock());

        sc.close();
    }
}
```

OUTPUT:



3.

AIM:

(For Advanced Learners – home) Create an AutoCloseable resource and use try-with-

resources to prove it always closes — even when an exception is thrown inside the block

— and that the original error is the one reported. Decide what to model as the resource.

CODE:

```
class MyResource implements AutoCloseable {
    MyResource() {
        System.out.println("Resource opened");
    }

    @Override
public void close() {
```

```
        System.out.println("Resource closed");
    }
}

public class AutoCloseDemo {

    public static void main(String[] args) {        try
(MyResource resource = new MyResource()) {
System.out.println("Inside try block");
            throw new RuntimeException("Original exception occurred");
        } catch (RuntimeException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        System.out.println("Program continues");
    }
}
```

OUTPUT:

```
loseDemo.java } ; if ($?) { java AutoCloseDemo }
Resource opened
Inside try block
Resource closed
Caught: Original exception occurred
Program continues
```